GARUDA SoC Block I: Processor Core

Table of Contents

GARUDA SoC — Block I: Processor Core

Block-Level Design Specification

Document ID	AERO-GARUDA-DS-001
Block ID	U_CORE
Revision	1.0
Status	Released for RTL
Process Node	28 nm
Target Frequency	200 MHz core / 100 MHz peripherals
Parent Document	GARUDA SoC TRM v2.0
Author	Team AeroSoC
Date	June 2026

Table of Contents

---

1. Introduction

1.1 Purpose

This document specifies the GARUDA processor core, Block I of the GARUDA SoC. The core is a custom RV32IM implementation built from scratch — it is not derived from Ibex or any existing core. Its defining responsibility is to integrate the Domain-Specific Unit (DSU, Block II) directly into its execute stage and to fetch and access memory over the SoC's two-tier AHB-Lite / APB fabric.

This is a block-level design specification. It defines the external pin contract, the pipeline microarchitecture, the DSU issue interface, the dual AHB-Lite master interface, the M-mode privilege and CLIC-based trap model, hazard and forwarding behavior, and the verification hooks. The RTL team builds against this document.

1.2 Scope

This document covers the core top-module interface to memory, interrupts, and debug; the five-stage pipeline and all datapath/control blocks; the Custom-0 issue path to the DSU and its stall/flush/writeback contract; the instruction-fetch front-end including the prefetch buffer; M-mode CSRs, synchronous exceptions, and the CLIC interrupt interface; hazard detection, forwarding, and branch handling; and reset, clocking, and the dual AHB-Lite master protocol.

This document does **not** cover: the DSU internals (Block II — see AERO-GARUDA-DS-002); the bus interconnect, bridge, memories, DMA, or peripherals (Blocks III–XXIV); physical design; or firmware.

1.3 Conventions

Signal names are lowercase-with-underscores; buses carry their width, e.g. `haddr_i[31:0]`. Active-low signals carry an `_n` suffix. Module port directions use the suffix `_i` (input) and `_o` (output) at the core boundary only; internal signals are unsuffixed. Vectors are MSB-on-the-left, bit `[0]` is the LSB. Two's-complement signed arithmetic is the default. Hexadecimal is written `0x1234_5678`.

Requirement language: **shall** / **must** denotes a normative requirement; **should** denotes a recommendation; **may** denotes an option.

1.4 Reference Documents

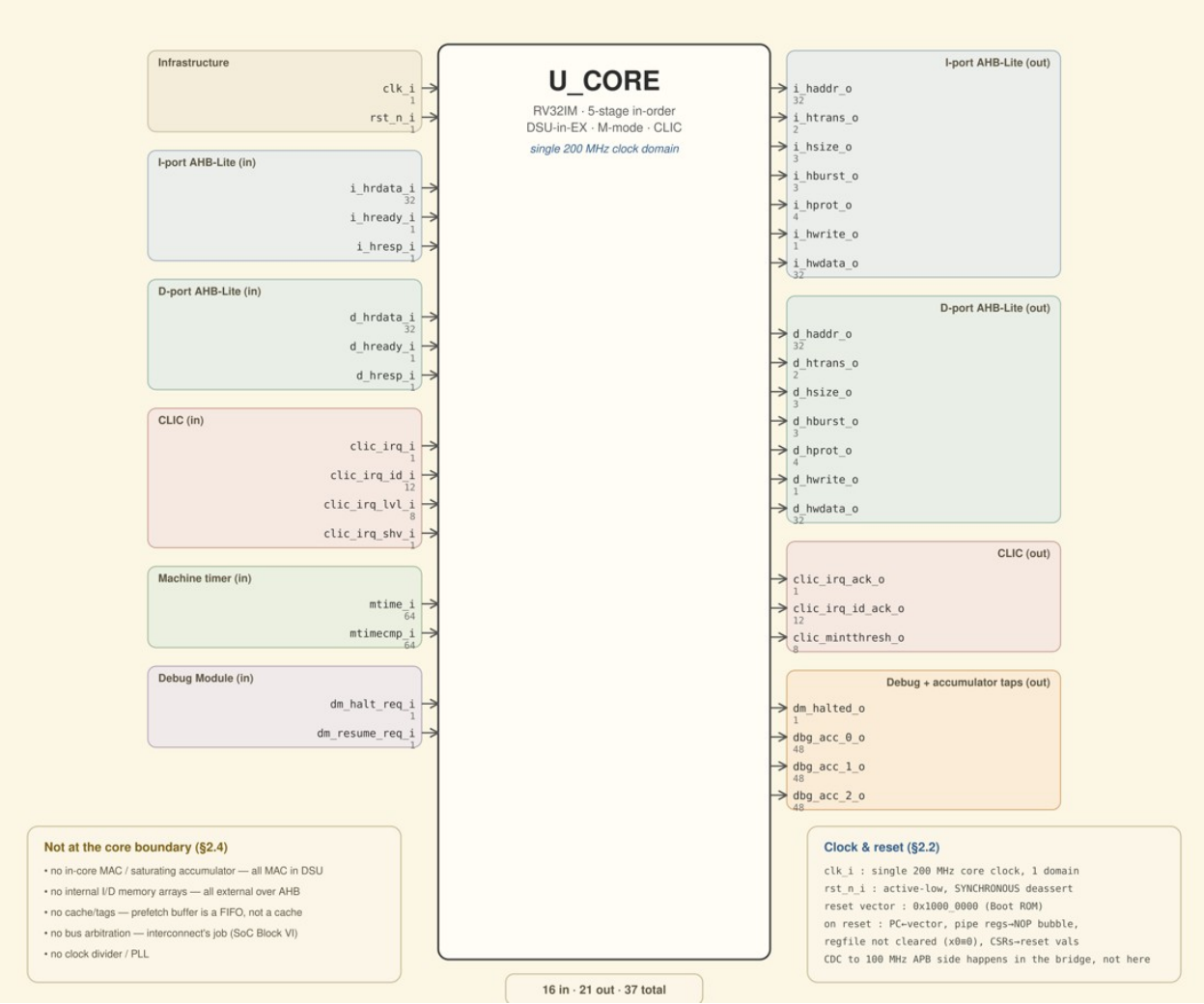
- GARUDA SoC Technical Reference Manual v2.0, Team AeroSoC, 2026.
- GARUDA SoC DSU Block-Level Design Specification, AERO-GARUDA-DS-002, Rev 1.0.
- The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, 20191213.

- The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, 20211203.
- RISC-V CLIC Specification (draft), used for the interrupt-controller interface.
- ARM AMBA 5 AHB Protocol Specification (IHI 0033), AHB-Lite subset.

---

2. Core as a Black Box (Top-Level Module)

The core is a single synthesizable unit (`U_CORE`). It presents two independent AHB-Lite master ports (Harvard: an instruction port and a data port), a CLIC interrupt interface, a RISC-V Debug-Module interface, the machine-timer inputs, and the global clock/reset. The DSU is instantiated **inside** the core in the execute stage and is therefore not visible at the core boundary — the only external evidence of the DSU is the three debug accumulator taps, which the core forwards to the Debug Module.



The complete signal-by-signal pin contract is given in Section 4. What does **not** exist at the boundary is listed in Section 4.4.

2.1 Clock and Reset

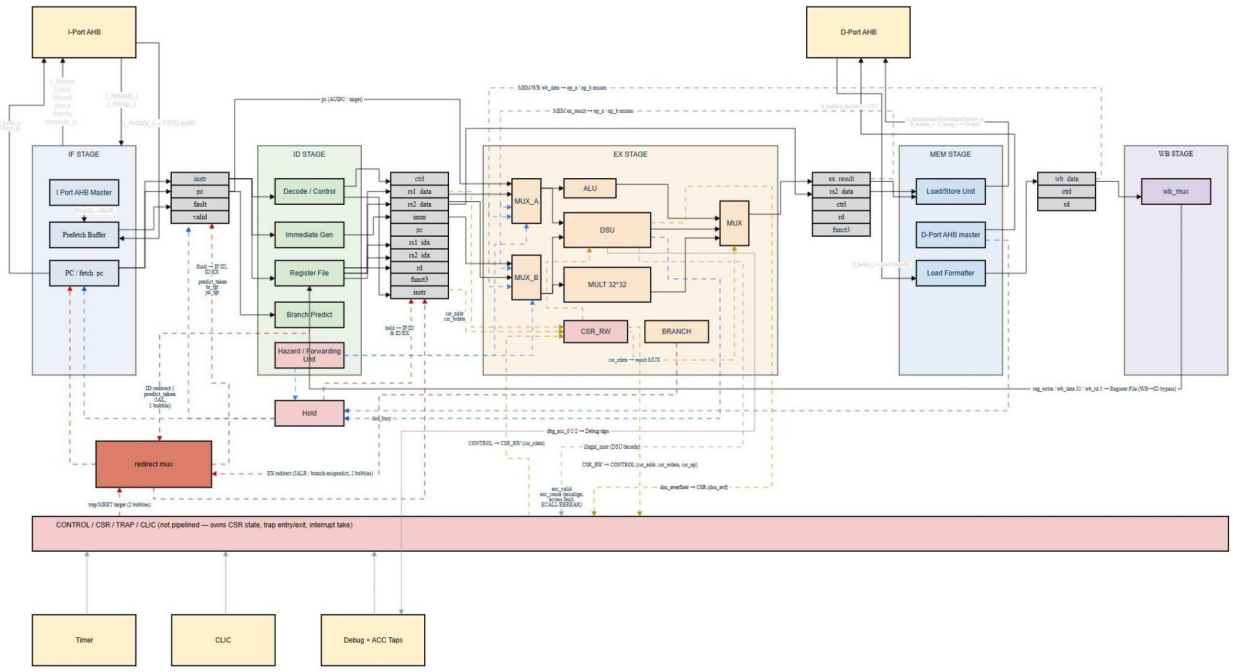
`clk_i` is the single 200 MHz core clock. The core is strictly single-clock-domain; all clock-domain crossing to the 100 MHz peripheral side happens in the AHB-to-APB bridge, outside the core.

`rst_n_i` is active-low with synchronous deassertion. On assertion: the program counter loads the reset vector (`0x1000_0000`, Boot ROM); all pipeline registers clear to the NOP bubble state; the register file is **not** cleared (`x0` reads as zero by construction); and all M-mode CSRs take their specified reset values.

---

3. Core Microarchitecture (Block Diagram)

The figure below is the authoritative **structural** view of the core: the five pipeline stages (IF, ID, EX, MEM, WB), the pipeline registers between them, the DSU embedded in EX, the dual AHB-Lite masters, and the non-pipelined CONTROL / CSR / TRAP / CLIC block that owns architectural CSR state and drives trap entry/exit.



3.1 Figure Legend

The block diagram uses the following line and box conventions:

Style	Meaning
Solid black line	Data bus / datapath

Style	Meaning
Solid gray line, slotted box	Pipeline register (`if_id`, `id_ex`, `ex_mem`, `mem_wb`)
Blue dashed line	Forwarding, hazard, and stall control (incl. `dsu_busy`, hold lines)
Red dashed line	Redirect and flush (branch / JALR / trap)
Gold / orange dashed line	CSR and DSU control & status (`csr_addr/wdata/op`, `csr_rdata`, `illegal_instr`, `dsu_overflow`, `csr_clear_overflow`)
Gray dashed line	Debug / accumulator taps (`dbg_acc_0/1/2`)

3.2 Scope of the Figure vs. This Document (Normative)

The block diagram shows datapath structure and major control flow. It is deliberately simplified for readability and does **not** draw every control qualifier or handshake pulse. The following fine-grained control semantics are specified normatively in the indicated sections, which take precedence over any visual simplification in the figure:

- **Operand-mux (MUX_A / MUX_B) select conditions** — §11 (Hazards & Forwarding) and §8 (Execute Stage).
- **Branch / jump target formulas and redirect timing** — §12 (Branch & Jump Handling).
- **EX result-mux priority and the `mem_to_reg` load override** — §8 (Execute Stage).
- **Exception priority and per-cause `mtval`** — §14 (Trap & Interrupt Architecture).
- **Interrupt-take gating and the CLIC acknowledge handshake** — §14.
- **Stall / flush composition and hold-vs-flush priority** — §11.
- **Full core↔DSU port contract** — §9 (DSU Issue Interface).

The figure is authoritative for **structure**; this document is authoritative for **behavior**. Where the two appear to differ, the text governs.

---

4. Top-Level Pin Contract

4.1 Pin Contract Table

Infrastructure

Signal	Dir	Width	Function
`clk_i`	in	1	200 MHz core clock.
`rst_n_i`	in	1	Active-low,

Signal	Dir	Width	Function
			synchronous-deassert reset.

Instruction AHB-Lite master port (I-port)

Signal	Dir	Width	Function
`i_haddr_o`	out	32	Instruction fetch address.
`i_htrans_o`	out	2	Transfer type (IDLE / NONSEQ / SEQ).
`i_hsize_o`	out	3	Always `3'b010` (word).
`i_hburst_o`	out	3	SINGLE or INCR (prefetch).
`i_hprot_o`	out	4	`0b0010` (privileged, opcode fetch).
`i_hwrite_o`	out	1	Tied `1'b0`; the I-port never writes.
`i_hwdata_o`	out	32	Tied 0.
`i_hrdata_i`	in	32	Fetched instruction word.
`i_hready_i`	in	1	Slave ready / wait-state.
`i_hresp_i`	in	1	OKAY(0) / ERROR(1); ERROR raises instruction access fault.

Data AHB-Lite master port (D-port)

Signal	Dir	Width	Function
`d_haddr_o`	out	32	Load/store address.
`d_htrans_o`	out	2	Transfer type.
`d_hsize_o`	out	3	Byte/half/word from `funct3`.
`d_hburst_o`	out	3	Always SINGLE

Signal	Dir	Width	Function
			`3'b000`.
`d_hprot_o`	out	4	`0b0011` (privileged, data).
`d_hwrite_o`	out	1	1 = store, 0 = load.
`d_hwdata_o`	out	32	Store data, lane-aligned.
`d_hrdata_i`	in	32	Load data.
`d_hready_i`	in	1	Slave ready / wait-state.
`d_hresp_i`	in	1	OKAY / ERROR; ERROR raises load/store access fault.

CLIC interrupt interface

Signal	Dir	Width	Function
`clic_irq_i`	in	1	Interrupt request pending (level).
`clic_irq_id_i`	in	12	Interrupt ID of the highest-pending source.
`clic_irq_lvl_i`	in	8	Interrupt level of that source.
`clic_irq_shv_i`	in	1	1 = selective hardware vectoring for this source.
`clic_irq_ack_o`	out	1	Core has taken the interrupt (1-cycle pulse).
`clic_irq_id_ack_o`	out	12	ID being acknowledged.
`clic_mintthresh_o`	out	8	Current interrupt threshold (from CSR).

Memory-mapped timer (FreeRTOS tick)

Signal	Dir	Width	Function
`mtime_i`	in	64	Machine time, from system timer block.
`mtimecmp_i`	in	64	Time-compare value (mirrored for the local timer-IRQ compare).

Debug-Module interface (RISC-V DM 0.13)

Signal	Dir	Width	Function
`dm_halt_req_i`	in	1	Halt request from Debug Module.
`dm_resume_req_i`	in	1	Resume request.
`dm_halted_o`	out	1	Core is halted in Debug Mode.
`dbg_acc_0_o`	out	48	DSU ACC FX tap (forwarded from DSU).
`dbg_acc_1_o`	out	48	DSU ACC FY tap.
`dbg_acc_2_o`	out	48	DSU ACC MAG tap.

4.2 Reset and Idle Behavior

Out of reset both masters drive `HTRANS = IDLE`. The I-port begins fetching from the reset vector once `rst_n_i` deasserts and `i_hready_i` is high. See §16 for the full AHB-Lite protocol.

4.3 Clock-Domain Notes

The core has exactly one clock domain. No core output is permitted to depend on a peripheral-domain clock. All synchronization to 100 MHz is external (AHB-to-APB bridge).

4.4 What Does Not Exist at the Core Boundary

The following are explicitly out of scope and **must not** be added without revising this specification:

- **No in-core MAC**, multiplier-accumulator, or saturating accumulator. All MAC math lives in the DSU.

- **No internal instruction or data memory arrays.** All memory is external over AHB-Lite.
- **No cache or tag logic.** The prefetch buffer is a FIFO, not a cache.
- **No bus arbitration.** Inter-master arbitration is the interconnect's responsibility (SoC Block VI).
- **No clock division or PLL.**

---

5. Pipeline Architecture

5.1 Overview

The core is a classic five-stage in-order pipeline: IF, ID, EX, MEM, WB. One instruction is issued per cycle in the absence of hazards. The DSU occupies the EX stage and can request a one-cycle hold via `dsu_busy`. There is no out-of-order completion and no register renaming.

Stage	Function
IF	Fetch from prefetch buffer (fed by I-port AHB master); PC generation.
ID	Decode, register-file read, immediate generation, hazard detection, static branch predict, JAL target.
EX	ALU, single-cycle multiply, DSU issue, branch/JALR resolution, address calculation, CSR read-modify-write.
MEM	Data memory access over the D-port AHB master; load alignment.
WB	Register-file writeback (ALU / load / multiply / DSU result).

5.2 Pipeline Registers

Four pipeline registers separate the stages: `if_id`, `id_ex`, `ex_mem`, `mem_wb`. Each supports three controls:

- **stall (hold):** retains current contents (used by `if_id` and the PC during a load-use or DSU hold).
- **flush (bubble):** forces the register's control fields to the NOP-equivalent (all write-enables low) on the next edge (used on branch mispredict and trap entry).
- **normal clocked update** otherwise.

On `rst_n_i` all four registers reset to the bubble state. The canonical NOP carried in a flushed bubble is `0x0000_0013` (`addi x0, x0, 0`).

Pipeline-register field contents (as drawn on the block diagram):

Register	Carried fields
`if_id`	`instr`, `pc`, `fault`, `valid`
`id_ex`	`ctrl`, `rs1_data`, `rs2_data`, `imm`, `pc`, `rs1_idx`, `rs2_idx`, `rd`, `funct3`, `instr`
`ex_mem`	`ex_result`, `rs2_data`, `ctrl`, `rd`, `funct3`
`mem_wb`	`wb_data`, `ctrl`, `rd`

5.3 Datapath Width and Register Model

All datapaths are 32-bit. The architectural register file is 32×32-bit (`x0`–`x31`), `x0` hardwired to zero. The file has two synchronous read ports (rs1, rs2) and one write port (WB). A third read port is **not** implemented — the DSU owns accumulator state internally and is never read back through the register file.

---

6. Instruction Fetch and Prefetch Buffer

6.1 Purpose

The fetch front-end decouples instruction supply from AHB-Lite fetch latency. It comprises the program counter, the prefetch buffer (a FIFO that runs ahead of the IF stage), and the I-port AHB-Lite master.

6.2 Program Counter and Fetch-Address Generation

Two address pointers exist:

- **`pc`** — the architectural program counter, the address of the instruction entering ID. It advances when the IF/ID register accepts a new instruction.
- **`fetch_pc`** — the speculative fetch pointer driving the I-port. It runs ahead of `pc` by up to the buffer depth, incrementing by 4 each time the I-port accepts a fetch (`i_hready_i & i_htrans_o != IDLE`) and the buffer is not full.

On a control-flow redirect (branch taken, JAL/JALR, trap, mispredict correction) **both** pointers are reloaded with the redirect target and the buffer is flushed (§6.5).

6.3 Prefetch Buffer Structure

The buffer is a depth-4 circular FIFO of 32-bit instruction words, each tagged with the PC it was fetched from (the tag is needed for trap `mepc` and for the branch target adder in ID).

- **Write side:** a word is written when the I-port returns valid read data (`i_hready_i` high in the data phase of an outstanding fetch) and the buffer is not full. The write enable is on the **memory side**.

- **Read side:** the IF stage pops one word per cycle when the buffer is non-empty and the pipeline is not stalled.
- **Occupancy:** maintained by `wr_ptr`, `rd_ptr`, and a wrap/occupancy counter giving `full` and `empty`. `full` backpressures `fetch_pc` advance; `empty` starves IF (inserts a fetch bubble, not a NOP into the architectural stream).

6.4 I-Port AHB-Lite Operation

The I-port issues word reads (`i_hsize_o = 3'b010`). Sequential fetches use `HTRANS = SEQ` with `HBURST = INCR` so the interconnect/SRAM can pipeline; the first fetch after a redirect is NONSEQ. `i_hwrite_o` is tied 0.

An `i_hresp_i = ERROR` on a fetch tags the corresponding buffer entry as faulting; the fault is raised as an **instruction access fault** synchronous exception when (and only when) that entry reaches ID, so a speculatively-fetched faulting word that is later flushed never traps. The `fault` bit travels in the `if_id` register and is consumed by the exception logic in the CONTROL block (§14.1).

6.5 Flush

On redirect, the whole buffer is invalidated in one cycle by resetting `wr_ptr`, `rd_ptr`, and the occupancy counter; any in-flight I-port fetch is allowed to complete on the bus but its returned data is dropped (not written). This clears the **entire** buffer, not a single slot.

---

7. Decode, Control, and Operands

7.1 Instruction Decode

The ID stage extracts `opcode[6:0]`, `rd[11:7]`, `funct3[14:12]`, `rs1[19:15]`, `rs2[24:20]`, `funct7[31:25]`, and the five immediate forms (I/S/B/U/J). It generates the full control bundle for the downstream stages: `reg_write`, `mem_read`, `mem_write`, `mem_to_reg`, `alu_src`, `alu_op`, `branch`, `jal`, `jalr`, `mul_en`, `dsu_en`, `csr_en`, `csr_op`, and `is_system`.

7.2 Supported Instruction Classes

Class	Opcode	Notes
LUI / AUIPC	`0110111` / `0010111`	U-type.
JAL / JALR	`1101111` / `1100111`	Link = PC+4.
BRANCH	`1100011`	BEQ/BNE/BLT/BGE/ BLTU/BGEU.
LOAD	`0000011`	LB/LH/LW/LBU/LHU.
STORE	`0100011`	SB/SH/SW.
OP-IMM	`0010011`	ADDI … SRAI.
OP	`0110011`	RV32I ALU + M-ext MUL family.

Class	Opcode	Notes
SYSTEM	`1110011`	CSRRW/S/C[I], ECALL, EBREAK, MRET, WFI.
Custom-0 (DSU)	`0001011`	Issued to the DSU; see §9.

M-extension MUL/MULH/MULHSU/MULHU are implemented in hardware (§8.2). DIV/DIVU/REM/REMU are **decoded but not executed**: they raise an illegal-instruction exception so the software Newton-Raphson handler runs.

7.3 Immediate Generation

A single combinational immediate generator selects the I/S/B/U/J form from `opcode` and sign-extends to 32 bits. B- and J-immediates are pre-shifted (LSB zero) and sign-extended; U-immediate is `{instr[31:12], 12'b0}`.

7.4 Register File

Synchronous-write, combinational-read. Read-during-write returns the new value via a **WB→ID bypass**, removing a structural hazard on back-to-back dependent ops. `x0` writes are discarded; `x0` reads return 0.

---

8. Execute Stage

8.1 ALU

A 32-bit ALU performing ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU. Operand A (via MUX_A) is the forwarded `rs1` **or** `pc` (for AUIPC and branch/jump target computation, which reuse the ALU adder). Operand B (via MUX_B) is the forwarded `rs2` **or** the sign-extended immediate, selected by `alu_src`. The forwarding selects on MUX_A/MUX_B are driven by the Hazard/Forwarding unit (§11.1).

8.2 Single-Cycle Multiplier

A combinational 32×32 signed/unsigned multiplier produces a 64-bit product within the EX cycle. `funct3` selects the returned half: MUL returns `product[31:0]`; MULH/MULHSU/MULHU return `product[63:32]` with the appropriate operand signedness. The multiplier has no internal pipeline registers and is the dominant EX-stage critical-path candidate.

Timing note. Closure at 200 MHz on the 28 nm PDK is *expected but not asserted* by this specification. The critical path is the full EX cone — the multiplier in series with the EX result mux (§8.3) — not the multiplier alone. If the path fails, the documented fallback is the TRM's 100 MHz fallback clock with no RTL change. No retiming is implied.

8.3 EX Result Selection

The value forwarded to MEM/WB as the EX result is selected, in strict priority order:

1. **csr_rdata**, when `csr_en` (CSRRW/CSRRS/CSRRC and immediate variants read the "old" CSR value into rd per §13.3 — this is the highest-priority input so the read-modify-write is never shadowed by a coincident DSU or multiply result on the same cycle);
2. **DSU result**, when `dsu_rd_valid` (DSU read/shift instructions);
3. **multiply result**, when `mul_en`;
4. **ALU result** otherwise.

Loads override this later in WB selection via `mem_to_reg` (§10.3). On the block diagram this priority is the EX-stage **MUX** fed by CSR_RW, ALU, DSU, and MULT 32×32; the CSR input is qualified by `csr_en` and the DSU input by `dsu_rd_valid`.

Erratum §6.3-E1 (resolved). An earlier draft's mux priority (§6.3) enumerated only DSU > MUL > ALU and omitted the CSR read-modify-write path required by §11.3 for CSRRW/CSRRS/CSRRC to return the old CSR value to rd. This is resolved per Option A of the erratum: csr_rdata is added as the highest-priority EX-result-mux input, gated by csr_en, ahead of the DSU and multiply results. This resolution is normative as of Rev 1.0 and the erratum is closed; architect sign-off recorded.

8.4 CSR Read-Modify-Write Datapath

`CSRRW/CSRRS/CSRRC` and their immediate variants execute in EX. The CSR_RW block in EX issues the access to the CONTROL/CSR block, which owns the architectural CSR state (CSRs are not pipelined). The write source into CSR_RW is `rs1_data` (register variants) or the 5-bit zero-extended immediate (immediate variants). The exchange is:

- `CSR_RW → CONTROL`: `csr_addr`, `csr_wdata`, `csr_op`.
- `CONTROL → CSR_RW`: `csr_rdata`, returned into the EX result mux as the highest-priority input (§8.3).

The custom `dsu_ovf` CSR (§13.2) is the only path that pulses `csr_clear_overflow` into the DSU and reads back `dsu_overflow`.

---

9. DSU Issue Interface

9.1 Integration Model

The DSU (`dsu_top`) is instantiated in the EX stage. The core decodes the Custom-0 opcode (`0001011`), presents operands and the **raw instruction word** to the DSU, honors the DSU's stall request, and routes the DSU's register result and exception back into the pipeline. The core does **not** interpret `funct5`, `acc_sel`, or the DSU immediate — the DSU decodes those internally.

9.2 Boundary Contract (Normative)

This boundary matches the released, compiling `dsu_top.v` RTL. It drives the full instruction word, not separated fields.

DSU port	Dir	Width	Driven by / consumed by the core
`clk`	in	1	`clk_i`.
`rst_n`	in	1	`rst_n_i`.
`flush`	in	1	**EX-stage** flush (branch mispredict or trap squashing the in-flight DSU op). Must be the EX flush, not the IF/ID flush.
`dsu_en`	in	1	`(opcode == 0x0B)` of the instruction in EX.
`instr`	in	32	The EX-stage instruction word (carries `funct5/acc_sel/imm/funct3/rd`).
`rs1`	in	32	Forwarded rs1 operand.
`rs2`	in	32	Forwarded rs2 operand.
`csr_clear_overflow`	in	1	Pulse from the CSR unit on a write to the `dsu_ovf` CSR.
`dsu_busy`	out	1	Stall request; asserted for one cycle on a same-accumulator dependent readback (§9.4). Drives the pipeline hold.
`dsu_rd_data`	out	32	Writeback value

DSU port	Dir	Width	Driven by / consumed by the core
			for MACREAD LO/HI and MACSHIFT.
`dsu_rd_valid`	out	1	Selects `dsu_rd_data` into the EX result mux and gates WB. Self-gated low during a `dsu_busy` hold.
`dsu_rd_addr`	out	5	Destination GPR for the DSU result.
`dsu_overflow`	out	1	Sticky saturation flag; exposed read-only in the CSR map.
`illegal_instr`	out	1	Reserved/illegal DSU encoding; OR-ed into the core's illegal-instruction exception.
`dbg_acc_0/1/2`	out	48	Accumulator taps; forwarded to `dbg_acc_*_o`.

> **Documentation note.** The DSU design spec (AERO-GARUDA-DS-002, Fig. II.1) draws separated inputs `funct5[4:0]`, `acc_sel[1:0]`, `imm[4:0]`. The released `dsu_top.v` instead takes the full `instr[31:0]` and emits `dsu_rd_addr`. **This specification follows the RTL, which is frozen.** The DSU design-spec figure should be updated to match; the core does not change.

9.3 Custom-0 Instruction Encoding (Reference)

The DSU decodes Custom-0 internally; the core does not. This table is provided so integrators and codegen can reason about the seam. Encoding is taken from the frozen `dsu_decoder.v` / `dsu_defs.vh`.

R-type form (`funct3 = 000`): `funct5 = instr[31:27]`, `acc_sel = instr[26:25]`, `rs2 = instr[24:20]`, `rs1 = instr[19:15]`, `rd = instr[11:7]`. I-type form (`funct3 = 001`, MACSHIFT): `acc_sel = imm[7:6]`, `shift_dir = imm[8]`, `shift_amt = imm[5:0]`.

Instruction	Form	funct5	Writes GPR?	Notes
MAC_SEL	R	0	No	rs1×rs2, accumulate into acc_sel.
MACSUB	R	1	No	rs1×rs2, subtract from acc_sel.
MACABS	R	2	No	MAC of absolute values.
MACDOT	R	3	No	Dual 16×16 dot-product accumulate.
MACLOAD	R	4	No	Load rs1 (sign-extended) into acc_sel.
MACCLEAR	R	5	No	Clear acc_sel.
MACSAT	R	6	No	Saturate acc_sel to int32 range (readback-class for hazards).
MACRD_LO	R	7	Yes	Lower 32 bits of acc_sel → rd.
MACRD_HI	R	8	Yes	Upper 16 bits of acc_sel → rd.
MACSHIFT	I	9	Yes	Arithmetic shift of acc_sel → rd.

acc_sel legal values are 00 (FX), 01 (FY), 10 (MAG). acc_sel = 11 is illegal and raises illegal_instr, as does any other reserved encoding or a bad funct3.

9.4 Stall and Writeback Timing (Normative)

The core treats `dsu_busy` as an **opaque one-cycle pipeline hold**: while `dsu_busy` is high, the PC and IF/ID register hold, and a bubble is inserted into MEM. The core makes no assumption about which instructions assert it — it obeys the signal.

When the DSU asserts `dsu_busy`. The DSU raises `dsu_busy` for exactly one cycle under a single condition: the immediately-preceding issued DSU op was a MACRD_LO, MACRD_HI, or MACSAT; **and** the current DSU op also reads an accumulator (MACRD_LO, MACRD_HI, MACSAT, or MACSHIFT); **and** both reference the same `acc_sel`. Compute ops (MAC_SEL, MACSUB, MACABS, MACDOT, MACLOAD, MACCLEAR) never assert `dsu_busy`. The hold history is cleared by `flush` and by reset.

Ownership (Normative). The hold-history state (whether the immediately-preceding DSU op was a readback on the current acc_sel) is internal to dsu_top.v and produces dsu_busy as its only externally visible effect. The core shall not maintain a duplicate or shadow copy of this history — it samples dsu_busy combinationally each cycle per the integration constraint above and reacts to it. Any core-side replication of DSU readback history is a specification violation even where it would be functionally redundant with the DSU's own logic.

Integration constraint (Normative). `dsu_busy` is a **registered** DSU output. The core's hold logic shall sample `dsu_busy` in the same EX cycle as the dependent instruction and assert the PC / IF-ID hold that cycle. The core **shall not** register `dsu_busy` a second time, or the inserted bubble lands one cycle late.

Writeback. For 1-cycle DSU read/shift instructions, the DSU asserts `dsu_rd_valid` in the same EX cycle, and the result is written to `dsu_rd_addr` in WB through the normal writeback path. Because `dsu_rd_valid = writes_regfile & ~dsu_busy` inside the DSU, it is forced low during a held cycle; the core's `ex_mem` capture of `dsu_rd_data` **shall** therefore be qualified by `dsu_rd_valid`, not by `dsu_en`. `dsu_en` compute instructions that do not write a register assert neither `dsu_rd_valid` nor `reg_write`.

Software ordering constraint (Normative, firmware/codegen). The DSU's two-cycle accumulate folds a compute op's products into the accumulator on the next write-enabling edge, and the stall FSM does **not** interlock a compute op against a following same-accumulator readback. Software **must not** issue a MACRD_LO/MACRD_HI/MACSAT/MACSHIFT that reads an accumulator in the cycle immediately following a compute op (MAC_SEL/MACSUB/MACABS/MACDOT) that writes the same accumulator; at least one independent instruction (or a compute op to the same accumulator) must separate them so the accumulate result is settled before it is read. This is a firmware/codegen contract, not a hardware interlock, and is verified by test C30.

9.5 Operand Routing and Forwarding

DSU operands take the same EX forwarding paths as ALU operands (§11.1): `rs1`/`rs2` presented to the DSU are the post-forwarding values. EX→EX and MEM→EX forwarding into a DSU source is therefore identical to forwarding into the ALU.

9.6 Exception and Flush Coupling (Normative)

`illegal_instr` from the DSU is treated exactly as a core-decoded illegal instruction: it traps with `mcause = 2`, `mepc =` the offending PC, and squashes the instruction.

On any EX flush (mispredict or trap), the core **shall** assert the DSU's `flush` so an in-flight 2-cycle MAC leaves accumulator state unchanged (the DSU clears its `sum_reg/carry_reg` and gates the accumulate write on `~flush`). The `flush` presented to the DSU **shall** be the EX-stage flush, not the IF/ID flush — feeding the wrong flush domain is a known integration trap. This satisfies the DSU sign-off item "trap during cycle 2 leaves the accumulator unchanged."

---

10. Memory Stage and Data Port

10.1 Load/Store Unit

The MEM stage drives the D-port AHB-Lite master. `d_hsize_o` is derived from `funct3`: byte (000), halfword (001), word (010). `d_hburst_o` is always SINGLE. Store data is lane-aligned per AMBA: a byte/halfword is replicated onto the correct lane of `d_hwdata_o`.

10.2 Alignment and Faults

The misalignment check completes **before** the D-port address phase commits (late EX / early MEM); on a misalignment the bus transaction is **not** issued.

Condition	Cause (`mcause`)
Halfword access to an odd address, or word access to a non-4-aligned address (load)	4 — load address misaligned
Same, store	6 — store/AMO address misaligned
`d_hresp_i = ERROR` on a load	5 — load access fault
`d_hresp_i = ERROR` on a store	7 — store access fault

10.3 Load Result Formatting

Load data from `d_hrdata_i` is extracted from the addressed lane and then sign- or zero-extended per `funct3` (LB/LH sign-extend; LBU/LHU zero-extend; LW passes through). The formatted value is selected into WB by `mem_to_reg`, overriding the EX result for load instructions.

10.4 Wait States

`d_hready_i` low extends the access: the MEM stage holds and propagates a stall upstream (PC, IF/ID, ID/EX hold; a bubble enters WB) until `d_hready_i` is high. Bank-parallel DSRAM access keeps the common case at zero wait states; same-bank CPU/DMA contention costs one cycle, resolved by the interconnect, transparent to the core.

11. Hazards, Forwarding, and Stalls

11.1 Forwarding

Two forwarding paths feed the EX operand muxes:

- **EX→EX:** the EX/MEM-stage result forwarded to the immediately following instruction.
- **MEM→EX:** the MEM/WB-stage result forwarded to an instruction two slots behind.

Each operand mux (MUX_A, MUX_B) selects, in priority order: the most recent producer (EX/MEM result), then the MEM/WB result, then register-file data (no hazard). The selects are driven by the Hazard/Forwarding unit. Forwarding write-enable qualification prevents forwarding from an instruction that does not write its destination (`reg_write` low, or `rd = x0`).

MUX_A data inputs: `rs1_data`, EX/MEM forward, MEM/WB forward, `pc` (AUIPC/target).
MUX_B data inputs: `rs2_data`, EX/MEM forward, MEM/WB forward, `imm`.

11.2 Load-Use Hazard

A load result is not available until the end of MEM. The hazard unit detects the load-use case by comparing the ID-stage `rs1_idx/rs2_idx` against the EX-stage `rd` when `mem_read` is set for the instruction in EX. On a hit it takes **two** simultaneous actions: it **stalls** PC and IF/ID, and it **flushes** ID/EX (inserts one bubble), so the value can be forwarded MEM→EX in the following cycle. This is the **only** data-hazard stall in the core.

11.3 DSU Hold

`dsu_busy` causes a one-cycle hold as specified in §9.4 (a same-accumulator dependent readback). Unlike load-use, a DSU hold **retains** the `id_ex` contents (the held DSU op must not be squashed) — it asserts hold lines but does **not** flush `id_ex`.

11.4 Stall / Flush Composition (Normative)

Hold sources combine onto the PC / IF-ID / ID-EX hold lines through a priority-OR: any asserted hold source holds the corresponding registers. The flush behavior, however, differs by source, so the combination is governed by the following rule when more than one event is active in the same cycle:

Hold-vs-flush priority table.

Event (this cycle)	PC	IF/ID	ID/EX	Notes
D-port wait state	hold	hold	hold	bubble enters WB; highest-

Event (this cycle)	PC	IF/ID	ID/EX	Notes
				priority hold.
DSU hold (`dsu_busy`)	hold	hold	**hold (no flush)**	held DSU op retained in `id_ex`.
Load-use	hold	hold	**flush**	one bubble into `id_ex`.
Branch mispredict (EX)	redirect	flush	flush	2 bubbles; redirect from EX.
Trap / MRET (CONTROL)	redirect	flush	flush	2 bubbles; redirect target from CONTROL.

Rule. A **redirect/flush from EX or CONTROL (mispredict, trap, MRET) takes precedence over any hold**: when a flush and a hold target the same register in the same cycle, the flush wins and the held op is squashed (it is being discarded anyway). A **DSU hold and a load-use stall cannot coexist** for the same instruction (an instruction is either a DSU op or a load consumer, not both); if a DSU hold overlaps a *downstream* load-use bubble, the load-use flush of `id_ex` still wins for `id_ex` while the PC/IF-ID hold is the OR of both. A **D-port wait state** outranks a DSU hold for the PC/IF-ID/ID-EX hold lines because the memory access must complete before any younger instruction advances.

11.5 Hazard Source Summary

Source	Penalty	Mechanism
Load-use	1 bubble	stall PC/IF-ID, flush ID/EX.
DSU same-acc readback	1 bubble	`dsu_busy` hold (§9.4); no ID/EX flush.
D-port wait state	N cycles	stall until `d_hready_i`.
I-port empty buffer	N cycles	fetch bubble, no architectural NOP.
Branch mispredict	2 bubbles	flush IF and ID (§12).
Trap / MRET	2 bubbles	flush IF and ID, redirect PC.

12. Branch and Jump Handling

12.1 Prediction Policy

Static prediction: backward branches (negative B-immediate) are predicted **taken**, forward branches predicted **not-taken**. The prediction is made in ID, where the B-immediate sign is available, and a predicted-taken branch redirects fetch from ID with a one-bubble penalty. For the APF inner loop (a backward branch iterating over neighbors) the prediction is correct on every iteration except the last.

12.2 Target Computation and Redirect Origin

Instruction	Target	Resolved in	Redirect origin	Penalty
Conditional branch	`PC + sext(Bimm)` (dedicated ID adder)	EX (direction)	ID (predicted) / EX (correction)	1 (predicted) / 2 (mispredict)
JAL	`PC + sext(Jimm)`	ID	ID	1 bubble; link = PC+4 → rd
JALR	`(rs1_fwd + sext(Iimm)) & ~1`	EX (needs register)	EX	2 bubbles; link = PC+4 → rd

JALR depends on a register, so it is resolved in EX with forwarded `rs1`; this is the path the prior draft omitted entirely. On the block diagram the **ID redirect** (predict_taken / JAL) and the **EX redirect** (JALR / branch-mispredict) are distinct arrows into the redirect mux, reflecting their different penalties.

12.3 Resolution and Correction

Conditional branches resolve in EX using the ALU comparison. The EX-resolved direction is compared against the ID prediction:

- **Prediction correct:** no action.
- **Predicted not-taken, actually taken:** flush IF and ID, redirect to the branch target.
- **Predicted taken, actually not-taken:** flush IF and ID, redirect to PC+4 of the branch.

Either misprediction costs two bubbles. JALR always costs two bubbles. All redirects flush the prefetch buffer (§6.5).

---

13. CSRs and Privilege

13.1 Privilege Model

M-mode only. There is no U- or S-mode. `misa` reports RV32IM with the **X** (non-standard) bit set for the Custom-0 DSU extension. WFI is implemented per §14.5.

13.2 Implemented CSRs

CSR	Address	Function
`misa`	0x301	ISA = RV32IM + X. Read-only.
`mvendorid`	0xF11	0 (non-commercial). Read-only.
`marchid`	0xF12	Implementation arch ID. Read-only.
`mimpid`	0xF13	Revision. Read-only.
`mhartid`	0xF14	0 (single hart). Read-only.
`mstatus`	0x300	MIE, MPIE, MPP (always 11). Global interrupt enable.
`mtvec`	0x305	Trap-vector base + mode[1:0]. Mode 3 = CLIC.
`mtvt`	0x307	CLIC trap-vector-table base (hardware vectoring).
`mie`	0x304	Interrupt-enable bits (legacy view; CLIC enables are per-source in the CLIC block).
`mip`	0x344	Interrupt-pending (read-only view).
`mepc`	0x341	Exception PC.
`mcause`	0x342	Trap cause; bit 31 = interrupt.
`mtval`	0x343	Faulting address or instruction.
`mscratch`	0x340	Scratch for trap handler.
`mintstatus`	0xFB1	Current and previous interrupt level (CLIC).

CSR	Address	Function
`mintthresh`	0x347	Interrupt-level threshold; drives `clic_mintthresh_o`.
`mnxti`	0x345	CLIC next-interrupt handler acceleration.
`mcycle/h`	0xB00/0xB80	Cycle counter.
`minstret/h`	0xB02/0xB82	Retired-instruction counter.
`dsu_ovf`	0xBC0	Custom: read DSU sticky overflow; write-1 pulses `csr_clear_overflow`.

13.3 CSR Access

CSRRW/CSRRS/CSRRC and their immediate variants execute in EX, read-modify-write atomically within the stage (datapath in §8.4). Writes to read-only CSRs and accesses to unimplemented CSRs raise illegal-instruction. The custom `dsu_ovf` CSR is the only path that drives `csr_clear_overflow` to the DSU and reads `dsu_overflow`.

---

14. Trap and Interrupt Architecture

The CONTROL / CSR / TRAP / CLIC block is **not pipelined**. It owns architectural CSR state, performs trap entry/exit, and drives the trap-target redirect into the redirect mux.

14.1 Synchronous Exceptions

The core detects the following synchronous exceptions, listed in **decreasing priority**:

Cause	`mcause`	Source
Instruction access fault	1	I-port ERROR on the fetched word reaching ID (`fault` bit in `if_id`).
Illegal instruction	2	Bad opcode/funct, DIV/REM, DSU `illegal_instr`, CSR violation.
Breakpoint	3	EBREAK.
Load address misaligned	4	Misaligned LH/LW.
Load access fault	5	D-port ERROR on a load.

Cause	`mcause`	Source
Store address misaligned	6	Misaligned SH/SW.
Store access fault	7	D-port ERROR on a store.
Environment call (M-mode)	11	ECALL.

Exceptions are **precise**: the offending instruction and all younger instructions are squashed; older instructions complete.

14.2 Per-Cause `mtval` (Normative)

Cause	`mtval` written
1 — instruction access fault	Faulting fetch PC.
2 — illegal instruction	The offending 32-bit instruction word.
3 — breakpoint	Address of the EBREAK instruction.
4 — load address misaligned	The misaligned load address.
5 — load access fault	The faulting load address.
6 — store address misaligned	The misaligned store address.
7 — store access fault	The faulting store address.
11 — ECALL	0.
Interrupts	0.

DIV/REM note. DIV/DIVU/REM/REMU (§7.2, decoded but not executed in hardware) share cause 2 with all other illegal-instruction sources; mtval is the offending 32-bit instruction word, not a special-cased value, so the software Newton-Raphson handler decodes funct3/rs1/rs2 directly from the trapped instruction rather than from a separate dispatch table.

14.3 CLIC Interrupt Interface and Take Condition (Normative)

The core implements CLIC-mode trap handling (`mtvec.mode = 3`). The external CLIC presents the highest-priority pending source on `clic_irq_i/_id_i/_lvl_i/_shv_i`.

Take condition. The core takes the presented interrupt when **all** of the following hold:

1. `clic_irq_i` is high, and
2. `mstatus.MIE` is set, and
3. `clic_irq_lvl_i` **>** `mintthresh` (strictly greater than the threshold), and
4. `clic_irq_lvl_i` **>** the current handling level in `mintstatus` (strictly greater than the level of any active handler).

Both level comparisons are strictly-greater-than. Threshold and active-level use the same > relation; a source at exactly the threshold or at the active level does **not** preempt.

On taking the interrupt the core:

1. pulses `clic_irq_ack_o` for one cycle with `clic_irq_id_ack_o` = the taken ID;
2. writes `mepc`, sets `mcause[31] = 1` with the interrupt ID, updates `mintstatus`;
3. if `clic_irq_shv_i = 1`, hardware-vectors through the `mtvt` table; otherwise jumps to `mtvec.base`;
4. flushes IF and ID and redirects fetch.

Acknowledge handshake. `clic_irq_ack_o` / `clic_irq_id_ack_o` are the core's only outputs back to the CLIC and form a one-cycle pulse handshake; the CLIC uses them to clear the pending/edge state of the taken source.

Preemptive nesting is supported through `mintstatus/mintthresh`: a higher-level source may interrupt an active handler. Nesting depth is bounded by the level count (8).

14.4 Trap Entry and Exit Sequence

On any trap (exception or interrupt): `mepc ← PC` of the trapped/next instruction; `mcause ← cause`; `mtval ←` faulting address/instruction per §14.2; `mstatus.MPIE ← MIE`; `mstatus.MIE ← 0`; `MPP ← 11`. MRET restores `MIE` from `MPIE`, restores the interrupt level, and redirects to `mepc`. Both entry and exit cost two bubbles (flush IF/ID). The trap/MRET target is driven from the CONTROL block into the redirect mux.

14.5 WFI Behavior (Normative)

WFI is implemented as a **pipeline-draining stall**:

1. On WFI decode, the core drains the pipeline to a precise boundary: all in-flight instructions older than the WFI complete (including any MEM access); no younger instruction is fetched.
2. The core then holds with the PC pointing at the **instruction after** the WFI.
3. The wake condition is **any pending, enabled interrupt** — specifically `clic_irq_i` high with the source enabled at the CLIC. The wake condition is evaluated **independently of `mstatus.MIE`**: WFI wakes even if `MIE = 0`. If `MIE = 1` and the take condition (§14.3) is met, the core takes the interrupt on wake; if `MIE = 0`, the core resumes execution at the instruction after WFI without taking the trap.
4. On wake, normal fetch resumes from the held PC.

Vectoring on wake (Normative). When the core takes an interrupt on WFI wake, entry proceeds exactly as in §14.3 steps 1–4, including the clic_irq_shv_i check: SHV sources hardware-vector through the mtvt table, non-SHV sources jump to mtvec.base. WFI wake is not a distinct trap-entry path — it is ordinary interrupt entry whose only WFI-specific precondition is that the wake check ignores mstatus.MIE (step 3 above), while taking the interrupt still requires MIE = 1 and the full §14.3 take condition. The core's WFI FSM and

CLIC entry logic shall invoke the same redirect path rather than implementing independent entry sequences.

14.6 Timer Interrupt

The FreeRTOS tick uses the machine timer: when `mtime_i ≥ mtimecmp_i` the local machine-timer-interrupt pending bit sets and is delivered through the CLIC like any other source. The timer block itself is external; the core only consumes the compare result.

---

15. Debug Interface

15.1 Debug Mode

The core supports RISC-V Debug 0.13 via an external Debug Module (DM) over JTAG. The core exposes `dm_halt_req_i`, `dm_resume_req_i`, and `dm_halted_o`. On halt request the core drains the pipeline to a precise boundary, enters Debug Mode (fetch redirected to the DM's program buffer / `dpc` held), and asserts `dm_halted_o`. Resume restores `dpc` and re-enters normal fetch. Four hardware breakpoints are supported via the trigger CSRs; a breakpoint match raises a debug entry rather than an EBREAK exception when Debug Mode is armed.

15.2 Accumulator Observability

The three DSU accumulator taps (`dbg_acc_0/1/2_o`, 48-bit each) are wired straight from the DSU to the core boundary so the DM can read accumulator state in a halted core without injecting MACREAD instructions, satisfying the DSU's built-in observability requirement.

---

16. AHB-Lite Master Interface

16.1 Two Independent Masters

The core presents two AHB-Lite master ports (I and D). Each is a compliant AHB-Lite master; neither performs arbitration. The interconnect (a separate SoC block) is responsible for multiplexing the I-port, D-port, and DMA onto the shared slaves and for generating per-slave HSEL and the read-data mux.

16.2 Transfer Protocol

- Address phase drives HADDR, HTRANS, HWRITE, HSIZE, HBURST, HPROT.
- Data phase samples HRDATA (read) or drives HWDATA (write), qualified by HREADY.
- HREADY low inserts wait states; the master holds address/control stable.
- HRESP = ERROR is the two-cycle AMBA error response; the master accepts both cycles, then raises the corresponding access-fault exception (instruction or load/store).

- The I-port may issue INCR bursts for sequential prefetch; the D-port issues only SINGLE.

16.3 Reset and Idle

Out of reset both masters drive `HTRANS = IDLE`. The I-port begins fetching from the reset vector once `rst_n_i` deasserts and `i_hready_i` is high.

---

17. Memory Map

The core targets the TRM memory map. Out-of-range accesses rely on the interconnect's default slave to return ERROR, which the core converts to an access fault.

Region	Base	Size	Port
Instruction SRAM	`0x0000_0000`	64 KB	I-port
Boot ROM	`0x1000_0000`	4 KB	I-port (reset vector)
Data SRAM	`0x2000_0000`	64 KB	D-port (4 banks)
Peripherals	`0x4000_0000`	—	D-port via APB bridge

The reset vector is `0x1000_0000` (Boot ROM). Instruction fetch normally runs in the ISRAM region after the bootloader copies firmware from external SPI flash.

---

18. Verification and Sign-Off

18.1 Built-in Observability

- `dbg_acc_0/1/2_o` — continuous DSU accumulator view.
- `minstret/mcycle` — retirement and cycle counts for IPC and trace checks.
- `dm_halted_o` and the trigger CSRs — breakpoint and single-step bring-up.

18.2 Required Directed Tests

ID	Scenario
C01	RV32I base ISA compliance (riscv-arch-test, RV32I suite).
C02	M-extension MUL/MULH/MULHSU/MULHU against reference products.
C03	DIV/REM raise illegal-instruction and reach the software handler.
C04	Load-use hazard: 1-bubble stall and

ID	Scenario
	correct MEM→EX forwarding.
C05	EX→EX and MEM→EX forwarding on dependent ALU chains.
C06	Static prediction: backward-taken loop, forward-not-taken if/else.
C07	Branch misprediction in both directions; 2-bubble flush correct.
C08	JAL link and target; redirect from ID, 1 bubble.
C09	JALR `(rs1+imm)&~1` target and link; 2 bubbles; forwarded rs1.
C10	LB/LH/LW/LBU/LHU sign/zero extension; SB/SH/SW lane alignment.
C11	Misaligned LH/LW and SH/SW raise causes 4/6; no bus transaction issued.
C12	D-port and I-port ERROR raise access faults 5/7/1 precisely.
C13	Speculatively-fetched faulting word that is flushed does not trap.
C14	Prefetch buffer fill/drain, full backpressure, empty fetch bubble.
C15	Whole-buffer flush on redirect; no stale instruction issued.
C16	DSU issue: `dsu_en` on Custom-0; rs1/rs2 forwarding into MAC_SEL.
C17	Same-accumulator dependent-readback hazard: a MACRD/MACSAT followed by a MACRD/MACSAT/MACSHIFT on the same `acc_sel` asserts `dsu_busy` for one cycle, inserts exactly one bubble, and suppresses `dsu_rd_valid` during the held cycle. Verify no stall for a different `acc_sel` or for compute ops.
C18	DSU read/shift writeback to `dsu_rd_addr` via WB.
C19	DSU `illegal_instr` traps with

ID	Scenario
	cause 2 and correct `mepc`.
C20	Trap during cycle 2 of a 2-cycle MAC leaves accumulator unchanged.
C21	All M-mode CSR read/write, illegal-CSR traps, MRET restore.
C22	ECALL/EBREAK traps; `mepc/mcause/mtval` correct per §14.2.
C23	CLIC interrupt take, ack handshake, vectoring (SHV and non-SHV).
C24	CLIC preemptive nesting honours `mintthresh/mintstatus`; strict-> compare at threshold and active level.
C25	Machine-timer interrupt drives the FreeRTOS tick at the programmed period.
C26	WFI drains and wakes on a pending enabled interrupt; wakes with `MIE = 0` without taking the trap.
C27	Debug halt/resume/single-step; accumulator taps readable while halted.
C28	Reset: PC = `0x1000_0000`, pipeline in bubble state, CSRs at reset values.
C29	FreeRTOS boot + context switch on the SoC testbench.
C30	Full APF inner-loop trace co-simulated against the DSU golden model; verifies the compute→same-acc-readback software ordering constraint (§9.4) is honoured.

18.3 Sign-Off Checklist

- ☐ RV32IM (less hardware divide) passes riscv-arch-test.
- ☐ DIV/REM trap path verified into a working Newton-Raphson handler.
- ☐ All forwarding and the single load-use bubble verified; no missed-forward cases.
- ☐ Branch prediction, misprediction recovery, JAL and JALR all correct.
- ☐ Prefetch buffer: corrected handshake, fetch-ahead, full/empty, whole-buffer flush.

- ☐ DSU issue boundary matches `dsu_top.v`; `dsu_busy` same-acc readback hold and writeback verified.
- ☐ DSU `illegal_instr` and trap-during-cycle-2 behaviour verified.
- ☐ `dsu_busy` sampled (not re-registered); `ex_mem` capture qualified by `dsu_rd_valid`; EX flush (not IF/ID) fed to DSU.
- ☐ Compute→same-acc-readback software ordering constraint documented and honoured by codegen.
- ☐ All synchronous exceptions precise; causes and `mtval` correct per §14.2.
- ☐ CLIC take/ack/vector/nesting verified; strict-> thresholds; timer tick correct.
- ☐ WFI drain/wake verified including the `MIE = 0` wake case.
- ☐ Both AHB-Lite masters compliant; ERROR two-cycle response handled.
- ☐ Debug halt/resume/step and accumulator observability verified.
- ☐ Synthesis closure at 200 MHz on the 28 nm PDK, or documented fallback to 100 MHz.
- ☐ FreeRTOS boots and runs the 1 kHz control loop on the SoC testbench.

End of document — AERO-GARUDA-DS-001 Rev 1.0.